Common Problems

Logging Service

By Evan King · Published May 8, 2026 · medium

Understanding the Problem

📝 **What is a Logger?**

A logger is the in-process library an application uses to record what's happening at runtime. Code calls `logger.info("user signed in")` from anywhere in the app, and the library timestamps the message, attaches the severity level, and writes it to one or more places like the console, a file, or both. Think Log4j, SLF4J, or Python's `logging` module. We're designing the library that lives inside one application, not a distributed log aggregation service.

Requirements

You sit down for the interview and the prompt comes in deliberately short:

> "Design a logging service. Or call it a logger, whichever you prefer."

Most of the design is hidden in what they didn't say. Spend the first few minutes pulling it apart before drawing anything.

Clarifying Questions

The first thing to nail down is what kind of logging system this is. "Logger" can mean wildly different things.

> **You:** "When you say 'logging service,' do you mean an in-process library the application links against? Or something that ships logs over the network to a central aggregator?"
> **Interviewer:** "In-process library. Network shipping, ingestion pipelines, and central aggregation are someone else's problem."

That answer cuts most of the design space. No queues, no schema registries, no fan-out across services. The deliverable is an object model that runs inside one application's process and writes to local destinations like stdout and files.

> **You:** "What severity levels should we support, and is there an ordering between them?"
> **Interviewer:** "DEBUG, INFO, WARN, ERROR, FATAL. Ordered from least to most severe in that order."

Five levels with a natural ordering. That's a finite set with no per-level behavior, which is a textbook enum. If you reach for a `Level` class hierarchy with `DebugLevel`, `InfoLevel`, and so on, you're doing too much.

> **You:** "Can a single logger write to multiple destinations at the same time? Like sending the same record to both the console and a file?"
> **Interviewer:** "Yes. That's the common case. A developer running locally wants logs in the console and also persisted to a file for later inspection. Each call should fan out to every configured destination."

Now you know "destination" is a first-class concept and that one log call hits all of them. That implies the library holds a list of destinations and iterates over them on every call.

> **You:** "Does each destination decide its own filter level, or is there one global level on the logger?"
> **Interviewer:** "Per destination. Each destination has its own minimum level. The console might want everything from DEBUG up, and the file destination might only care about WARN and above. Records below a destination's threshold should be dropped before being written."

This rules out putting the level filter on the logger and pushes it down to the destination. It also means the same record gets evaluated independently by each destination, which is fine because the record itself is immutable once created.

> **You:** "And the format the records are written in. Is that fixed, or does it vary?"
> **Interviewer:** "Varies. Sometimes plain text, sometimes JSON. And the format is independent of the destination type. You should be able to write JSON to the console, plain text to a file, or any combination."

This is the requirement that shapes the class model. If format and destination were coupled, you'd end up with a class for every (format, target) pair like `JsonFileDestination`, `PlainConsoleDestination`, and so on. Add a third format and a third target and you have nine classes. Since the requirement says they vary independently, the right move is to compose them instead of multiplying classes.

> When a requirement gives you two dimensions that vary independently, that's almost always a signal to use composition over inheritance. Two interfaces composed together let you mix any combination without writing N×M classes. Watch for these axes-of-variation hints in any LLD prompt.

> **You:** "What about concurrency? Multiple threads in the same app are going to be calling `log()` simultaneously. What's the expectation?"
> **Interviewer:** "Thread-safe. Each record's bytes have to land on a destination atomically — one record's bytes can't be split across or mixed with another's. For a single thread, records appear in call order. Across threads, no strict ordering beyond each record's timestamp."

Concurrency is in scope, so locking is part of the design, not a cleanup pass at the end. Per-record atomicity is the bar — two threads racing to a stdout buffer can't smear one record's bytes across another's. Strict global submission order across threads is a harder requirement that would push toward queues and single-writer threads, and they're not asking for that.

> **You:** "Last one. Is configuration static, set at startup, or do we need to handle hot-reloading destinations and levels at runtime?"
> **Interviewer:** "Static. Configured once at startup. Hot-reload, async or buffered writes, log rotation, and network destinations are all out of scope, though the design should not block adding a remote destination later."

The last clause is the one that shapes the design. You don't build remote destinations now, but the model needs an extension point so adding one later doesn't force a rewrite of `Logger`. As long as destinations are pluggable behind an interface, you're fine.

Final Requirements

After that back-and-forth, you'd write this on the whiteboard:

FINAL REQUIREMENTS

```
Requirements:
1. Five severity levels: DEBUG < INFO < WARN < ERROR < FATAL.
2. Each record carries timestamp, level, message, emitting thread name.
3. Logger writes each record to one or more destinations, set at startup.
4. Each destination has its own min-level threshold and its own format.
   Format and destination type vary independently.
5. Concurrent calls are safe. A record's bytes never interleave with
   another record's bytes on the same destination.


Out of scope:
- Hot-reloading config at runtime
- Async / buffered writes
- Remote / network destinations in v1 (design should accommodate)
- Hierarchical / named loggers (com.app.service inheriting from com.app)
```

> Notice we explicitly scoped out async writes, hot-reload, and remote destinations. Each of those is a real production concern, but each is a layer that sits on top of the core object model rather than being part of it. Calling them out by name signals that you considered them and chose not to build them, which reads very differently from forgetting they exist. Most of them come back as natural extensions in the follow-up section anyway.

Core Entities and Relationships

With requirements pinned down, the next step is figuring out what objects make up the system. Look for nouns in your requirements, but don't turn every one into a class. Some are fields on other classes. Some are enums. Some are just strings passed between methods. The filter is whether the noun owns state, enforces a rule, or has its own lifecycle. If it doesn't, it doesn't deserve a class.

Let's walk through the candidates:

The application — not an entity. The thing calling `logger.info("...")` lives outside our system. We don't model it. Same goes for threads. The OS owns those. We just read the current

thread's name at the call site and put it on the record we're building.

Severity level / timestamp / message — fields, not entities. None of these own state or enforce a rule. The level is one of five fixed values, the timestamp is a primitive, the message is a string. Each is a field on something else. The interesting question is what they're a field on, which leads to the first real entity.

LogRecord — entity. Every call to `log()` generates a unit of data made up of a timestamp, a level, a message, and a thread name. These fields always travel together. Every destination receives them, every formatter serializes them, and they never change after creation. That's the textbook case for a value object. Modeling them as one class instead of passing four parameters everywhere gives formatters a stable shape and lets you add fields later (a logger name, a request ID, a context map) without changing every method signature in the system.

Logger — entity. Something has to be the public face of the library. When the application calls `logger.info("...")`, something has to capture the timestamp and thread name, build a `LogRecord`, and dispatch it to every configured destination. That's `Logger`. It owns the list of destinations and exposes `log()` plus the convenience helpers (`debug`, `info`, `warn`, `error`, `fatal`) that callers actually use. It's the entry point and the only class the application ever touches.

Destination — entity. Each output target (console, file, future remote) needs three things. A minimum level threshold to filter on. A format to serialize the record. And the actual write mechanism. That's enough state and behavior to deserve a class of its own. It's also the natural home for the per-destination lock, since synchronization belongs with the resource being protected.

Formatter — entity. The "format and destination type are independent" requirement decides this one. If `Destination` did formatting itself, you'd need a separate destination class for every (format, target) pair. That's the 2D class explosion. Pulling format into its own interface means a single `Destination` composes a formatter and a write target, and any combination is just a constructor argument. Two implementations exist now (plain text, JSON) and more are real possibilities, so the interface earns its place.

After filtering, we're left with these:

Entity	Responsibility
Logger	The orchestrator. Holds the immutable list of destinations, exposes `log()` and convenience methods, captures per-call data (timestamp, thread name) and builds the `LogRecord`.
Destination	One configured output target. Owns its minimum level threshold, holds a reference to its formatter, and serializes the filter-format-write workflow. Where the per-destination lock will live.
Formatter	An interface for serializing a `LogRecord` to a string. Two implementations exist (plain text and JSON), and new formats become new implementations without touching anything else.
LogRecord	An immutable value object carrying the four pieces of per-call data (timestamp, level, message, thread name). Created in `Logger.log()`, consumed by every destination.

Beyond those four, one enum rounds out the model. `LogLevel` is the five-valued enum from the requirements, with a natural ordering. It shows up on both the record's severity and the destination's threshold. No per-level behavior, no class-per-level. A `DebugLevel` / `InfoLevel` hierarchy is the classic over-modeling mistake; we're not doing that.

The relationships are simple. `Logger` holds many `Destination` instances, fixed at construction. Each `Destination` holds exactly one `Formatter`. `Logger` creates a `LogRecord` per `log()` call and hands the same record to every destination. Each destination independently checks the level, formats the record (or doesn't, if filtered out), and writes. No back-references, no cycles, no shared mutable state outside each destination's own resource.

> You'll notice we haven't introduced an abstraction for "the thing that actually writes bytes" yet. That's deliberate. It's natural to look at `ConsoleDestination` and `FileDestination` and want to factor out a write target up front, but the cleanest version of that abstraction only becomes obvious once we work through the inheritance-vs-composition tradeoff during class design. Forcing it into the entity stage skips the inheritance-vs-composition reasoning, which is the most useful piece of this section.

Class Design

With our four entities identified, it's time to define their interfaces. What state does each one hold, and what methods does it expose?

We'll work top-down, starting with `Logger`, then handling the data types (`LogRecord`, `LogLevel`), then `Formatter`, and finishing with `Destination`. `Destination` is where most of the interesting design decisions live.

For each class, we'll ask two questions:

1. What does this class need to remember to satisfy the requirements (its state)?
2. What operations does it need to support (its methods)?

Logger

`Logger` is the orchestrator. The application calls `logger.info("...")`, and `Logger` is the only class the application ever needs to know about. Everything else lives behind it.

From the requirements:

Requirement	What `Logger` must track
"Logger writes each record to one or more destinations, configured at application startup"	The list of destinations

That's it. `Logger` has exactly one field:

LOGGER STATE

```
class Logger:
    - destinations: List<Destination>   // immutable after construction
```

Why `destinations` is immutable after construction. Config is set once at startup, so the list never changes. Iteration is safe under concurrent calls with no locking. A mutable list with

`addDestination()` would force locking around iteration without buying anything the requirements asked for.

Now the operations:

Need from requirements	Method on `Logger`
"Threads can emit log messages at one of five levels"	`log(level, message)` builds a `LogRecord` and dispatches
Convenience for the five levels	`debug`, `info`, `warn`, `error`, `fatal`, each delegating to `log`

LOGGER

```
class Logger:
    - destinations: List<Destination>

    + Logger(destinations: List<Destination>)
    + log(level: LogLevel, message: String)
    + debug(message: String)
    + info(message: String)
    + warn(message: String)
    + error(message: String)
    + fatal(message: String)
```

The constructor takes the destinations once and stores an immutable copy. There's deliberately no `addDestination`, no setters, no builder. Config is fixed at startup, so the API doesn't expose a way to mutate it. The convenience methods are trivial delegation. `info(msg)` just calls `log(INFO, msg)`. They're worth including because they match the API callers actually use. Calling `log(LogLevel.INFO, "...")` everywhere works, but it's noisier than `info("...")`, and a logger is infrastructure that tens of thousands of call sites will touch so the ergonomics matter.

The interesting part of `log()` is what it captures and what it doesn't. Timestamp and thread name come from the calling thread at the moment `log` runs (`now()` and `Thread.currentThread().getName()`, or the equivalent in your language). Both go into a freshly built `LogRecord`, which is handed to every destination. The level and message come from the

caller. No state on `Logger` mutates. Iteration over destinations is sequential and runs on the calling thread. No fan-out across worker threads, because async dispatch is out of scope.

LogRecord

`LogRecord` is the value object flowing through the system. Every `log()` call creates one, every destination consumes one, and the fields never change after construction. It's a dumb container by design — no behavior, just data, packed tightly so a record is cheap to allocate on the hot path.

Requirement	What `LogRecord` must track
"Each log record carries a timestamp, level, message, and thread name"	All four fields

LOGRECORD

```
class LogRecord:
    - timestamp: Instant
    - level: LogLevel
    - message: String
    - threadName: String

    + LogRecord(timestamp, level, message, threadName)
    + getters for all fields
```

Why all fields are read-only. A record represents what happened at one moment in one thread. None of those facts change after the call site. Making the fields immutable closes off a whole class of bugs. Any destination, any formatter, any future extension can read a record without worrying about racing another thread or accidentally mutating shared state. For value objects, immutable fields are the default. Here we don't even have collections to defensively copy, just primitives, an enum, and a timestamp.

Why this is a class and not four parameters. It would be tempting to skip `LogRecord` entirely and have `Logger.log()` pass `(timestamp, level, message, threadName)` to every destination. That works for v1 but rots fast. The day you add a logger name, a request ID, or a

context map, every method signature in the system has to change. Grouping the data into a record means adding a field touches one class definition and the orchestrator that builds the record, instead of changing every signature on `Destination` and `Formatter`. This is the same reason `Ticket` is its own class in the **Parking Lot** breakdown. Both are immutable records that group data the orchestrator manages.

> The pseudocode shows getters for clarity, but in a modern language this collapses to one line. Java records, Kotlin data classes, Python `@dataclass(frozen=True)`, TypeScript `readonly` fields, Go structs with unexported fields. All of them give you immutable records without the Java-from-2005 ceremony.

Formatter

`Formatter` is an interface for serializing a `LogRecord` to a string. Two implementations exist now (plain text, JSON) and the extension axis is real (CSV, key-value, XML, language-specific structured formats), so the interface earns its keep. This is the **Strategy pattern** — pulling format behind its own interface lets a `Destination` compose whichever formatter it wants, and adding a CSV formatter tomorrow is one new class with zero changes to anything that already exists.

FORMATTER

```
interface Formatter:
    + format(record: LogRecord) -> String

class PlainTextFormatter implements Formatter
class JsonFormatter implements Formatter
```

Formatters are pure functions. They take a record, return a string, end. That makes them safe to share across threads and across destinations. Two destinations can hold a reference to the same `JsonFormatter` instance without any synchronization, because there's nothing to synchronize.

Keeping formatting behind its own interface (rather than as a method on `Destination`) is the entire reason this design avoids the 2D class explosion. Two formats × two destination types

would be four classes if format and destination were coupled. Pulling format into its own type means one `Destination` class composes a formatter, and any combination is a constructor argument. The formatter owns serialization. The destination owns the write. Neither knows the other's internals.

Destination

`Destination` is the most fun class to design. It owns a minimum level threshold for filtering, a formatter for serializing, and the actual write mechanism for output. It also owns the per-destination lock for concurrent safety.

A candidate's first pass at `Destination` typically takes one of two shapes. The first is a single concrete class that branches on a `type` field inside its `write()` method:

```
class Destination:
    - formatter, minLevel, type, filePath
    + write(record):
        if type == CONSOLE: ...
        else if type == FILE: ...
```

The second is an inheritance hierarchy with abstract `Destination` and `ConsoleDestination` / `FileDestination` subclasses. Both are reasonable starting points, and either one can be made to work. The difference between the two, plus the third option that beats them both, is worth a real discussion.

Bad Solution: Single class with type branching

Good Solution: Inheritance with template method

Great Solution: Composition with a Sink interface

We're going with the composition variant. It keeps the filter-and-format invariant in one place (the same win as the inheritance approach), avoids a hierarchy that pays no real dividend, and

gives the requirement-stated remote destination a clean place to land as a new `Sink`. Composition wins over inheritance unless inheritance is genuinely earned, and here it isn't.

`Destination` itself stays concrete. There's only one valid filter-format-lock-delegate shape, and variation lives behind the `Sink` and `Formatter` interfaces, not in `Destination`. Abstraction earns its place. Adding an `IDestination` interface for one implementation would be indirection for its own sake. This is **Dependency Inversion** used where it matters. The high-level workflow class depends on `Sink` and `Formatter` abstractions, not on `ConsoleSink` or `JsonFormatter`.

> Inheritance variant is also defensible in an interview. The senior signal is articulating the choice, not memorizing one answer. If you walk through both options and pick inheritance for its simplicity at small scale, you'll get credit. The design fails only if you can't explain why you didn't pick the other one.

So our `Destination` is one concrete class composing a formatter, a level threshold, and a sink.

DESTINATION

```
class Destination:
    - formatter: Formatter
    - minLevel: LogLevel
    - sink: Sink

    + Destination(formatter, minLevel, sink)
    + write(record: LogRecord)
```

Sink

`Sink` emerged from the inheritance-vs-composition discussion. It's a small interface, one method and one responsibility, and it's where future extensions will plug in.

SINK

```
interface Sink:
    + write(formatted: String)
```

```
class ConsoleSink implements Sink
class FileSink implements Sink:
    - filePath: String
```

`ConsoleSink` writes to stdout. `FileSink` opens (or appends to) the file at `filePath` and writes. Both are dumb on purpose. They don't filter, they don't format, they don't lock. Those concerns live one layer up in `Destination`. A `Sink` just turns a string into bytes on a target. That narrowness is what makes the requirement-stated future remote destination drop in cleanly. A `RemoteSink` that writes to a network endpoint is a new `Sink` implementation, no other class in the system has to change.

Final Class Design

That's the complete model. `Logger` orchestrates the dispatch. `LogRecord` is the immutable unit of data. `LogLevel` is the five-valued enum. `Formatter` is the serialization interface, `Sink` is the output interface. `Destination` composes them. The pieces fit together with no cycles, no shared mutable state across classes, and a clear extension axis (new sinks, new formatters) that doesn't require touching `Logger`.

FINAL CLASS DESIGN

```
enum LogLevel:
    DEBUG
    INFO
    WARN
    ERROR
    FATAL
    // ordered: DEBUG < INFO < WARN < ERROR < FATAL

class LogRecord:
    - timestamp: Instant
    - level: LogLevel
    - message: String
    - threadName: String

    + LogRecord(timestamp, level, message, threadName)
    + getters for all fields

interface Formatter:
    + format(record: LogRecord) -> String
```

```
class PlainTextFormatter implements Formatter
class JsonFormatter implements Formatter

interface Sink:
    + write(formatted: String)

class ConsoleSink implements Sink
class FileSink implements Sink:
    - filePath: String

class Destination:
    - formatter: Formatter
    - minLevel: LogLevel
    - sink: Sink

    + Destination(formatter, minLevel, sink)
    + write(record: LogRecord)

class Logger:
    - destinations: List<Destination>

    + Logger(destinations: List<Destination>)
    + log(level: LogLevel, message: String)
    + debug(message: String)
    + info(message: String)
    + warn(message: String)
    + error(message: String)
    + fatal(message: String)
```

The design demonstrates **Separation of Concerns** all the way through, with orchestration in `Logger`, data in `LogRecord`, classification in `LogLevel`, serialization in `Formatter`, output in `Sink`, and the per-destination invariants (filter, compose) in `Destination`. Every class owns exactly one reason to change. That's **Single Responsibility** the way it actually matters in practice, not "tiny classes" but "one axis of change per class." Adding a new format, a new sink, or a new destination configuration touches one place, not the entire model.

Implementation

With the class design locked in, we need to implement the actual method bodies. Before diving in, check with your interviewer. Some want working code in a specific language, others prefer

pseudocode, and some just want you to talk through the logic. We'll use pseudocode here. It's the most common interview output and it lets us focus on the design choices rather than language-specific syntax.

For each method, we'll follow a pattern:

1. **Define the core logic** - The happy path that fulfills the requirement
2. **Handle edge cases** - Invalid inputs, boundary conditions, unexpected states

Interviewers usually focus on the most interesting methods. For our logger, those are:

- `Logger.log()` - shows how per-call data gets captured and fanned out to destinations
- `Destination.write()` - shows the filter-format-lock-write pipeline and how to handle a failing sink

Logger

`Logger.log()` is the entry point for every call into the library. The body is short — four lines of pseudocode — but the sequencing of those lines matters.

Core logic:

1. Capture the current timestamp.
2. Capture the current thread's name.
3. Build a `LogRecord` from those plus the caller's level and message.
4. Iterate over destinations and call `write(record)` on each.

Edge cases:

- Null or empty message
- A destination's `write()` throws

LOGGER.LOG

```
log(level, message):
    record = new LogRecord(
        timestamp  = now(),
        level      = level,
        message    = message,
        threadName = currentThread().name
```

```
        )
        for destination in destinations:
            destination.write(record)
```

`now()` and `currentThread().name` get called once at the top of `log()` rather than once per destination, so every destination sees the same record with the same timestamp and the same thread name. That matches the requirement exactly. One call fans out, and every destination filters and writes the same data independently. There's also no level filtering at the `Logger` level, because the threshold is a per-destination concern in our design, and the filter has to live wherever the threshold lives. And there's no locking around the iteration itself. The `destinations` list is immutable after construction, so concurrent threads can walk it without stepping on each other, and the synchronization that the requirements actually demand lives one layer down inside `Destination.write()`, next to the shared resource it's protecting.

On the edge cases, a null or empty message gets accepted as-is rather than guarded against. The logger isn't in the business of validating what callers hand it, and bolting a defensive null check onto a code path that fires from tens of thousands of call sites adds noise everywhere for almost no real protection. The right move is to pick a stance, say it out loud in the interview, and keep going. The more interesting case is what happens when one of the destinations throws partway through the iteration. We don't want a flaky file destination to silently swallow the same record that would have made it to the console, and we definitely don't want `Logger.log()` itself to start throwing back to the caller. The fix lives one layer down, inside `Destination.write()`, which catches its own failures so the loop in `Logger.log()` never sees them. The Destination section walks through exactly how that works.

> One temptation is to "speed up" the iteration by fanning out across worker threads, one per destination, so a slow file write doesn't block the console write. Don't. The destinations are already independent (each has its own lock), so the only thing parallel iteration buys you is faster latency on `log()` itself. The cost is a thread per call, lifecycle management, and (depending on your language) reordering guarantees that get harder to reason about. If async dispatch matters, that's a separate decision covered in the concurrency DeepDive, and it doesn't live in `Logger.log()`. Keep this loop sequential and dumb.

The convenience methods are one-liners. They exist purely so callers can write `info("...")` instead of `log(INFO, "...")` at every call site:

LOGGER CONVENIENCE METHODS

```
debug(message): log(DEBUG, message)
info(message):  log(INFO, message)
warn(message):  log(WARN, message)
error(message): log(ERROR, message)
fatal(message): log(FATAL, message)
```

Simple!

> Capturing the timestamp at the top of `log()` rather than later in `Destination.write()` is a small but real decision. If you captured it inside the destination, two destinations would record slightly different timestamps for the same record (different by microseconds, but still). Capturing once at the call site means every destination sees the same moment, which is what callers expect when they read a log file later.

Destination

`Destination.write()` filters, then formats, then writes to the sink under a lock. Two interesting decisions show up in the body — the locking strategy, and what to do when the sink throws.

Core logic:

1. Drop the record if its level is below the destination's threshold.
2. Format the record using the formatter.
3. Acquire the per-destination lock.
4. Hand the formatted string to the sink.
5. Release the lock (always, even if the sink throws).

Edge cases:

- Level below threshold

- Lock contention
- Sink throws on write

Below-threshold records get a silent drop — no error, no diagnostic, that's the whole point of a threshold. Lock contention just blocks the calling thread until the lock is free; v1 doesn't bother with timeouts. The interesting case is the sink throwing, which gets its own DeepDive below. First we need to settle where the lock lives, because that decision shapes the rest of `write()`.

Requirement 5 says concurrent calls must be safe and a record's bytes must not interleave with another record's on the same destination. That's a correctness problem — the file handle and the stdout buffer are shared state that two threads can corrupt by racing each other. Class design pointed at a per-destination lock — the destination owns the resource, so it owns the lock. The first instinct is usually coarser, like wrapping `synchronized` around `Logger.log()`. Two options worth walking through to motivate the choice.

Bad Solution: Global lock on Logger.log()

Good Solution: Per-destination lock around the sink write

Per-destination lock around `sink.write` is the right default. It's the same per-resource locking pattern that shows up in **BookMyShow**'s per-showtime seat reservations and **Inventory Management**'s per-warehouse stock. Each shared resource gets its own lock, and the orchestrator (here, `Logger`) never holds a single global lock across all of them. That's what lets a slow file write coexist with an instant console write without one blocking the other.

> It's tempting to lift the lock up to `Logger.log()` because "thread safety on the logger" sounds like the right framing. It usually isn't. The shared state lives on each destination's sink, not on the logger itself, so by default the lock belongs next to the sink. When the shared state lives somewhere else, that's a strong signal the lock belongs there too.

If an interviewer pushes on caller latency, batching, or strict per-destination sequencing, the natural next step is async writes per destination. We cover that as an extension below (**How would you make `log()` non-blocking?**). For v1, the per-destination lock is enough.

With locking settled, here's the actual method body:

DESTINATION.WRITE

```
write(record):
    if record.level < minLevel:
        return                                  // silent drop

    formatted = formatter.format(record)    // outside the lock

    lock.acquire()
    try:
        sink.write(formatted)
    catch e:
        // see DeepDive below
        ...
    finally:
        lock.release()
```

That leaves the `catch` block. What goes there when `sink.write()` throws?

Bad Solution: Let it propagate

Good Solution: Catch and swallow inside Destination

Great Solution: Swallow but emit a diagnostic to a fallback stream

The "great" option is the right default for production. For interview scope, "good" is a defensible answer - swallow the exception and move on. Mention you're aware of the silent-failure problem and would add a stderr diagnostic in a real implementation, then keep going.

Formatter implementations

Formatters take a `LogRecord` and return a string. They're pure functions, which is what makes them safe to share across destinations and across threads with no synchronization.

PLAINTEXTFORMATTER.FORMAT

```
format(record):
    return record.timestamp + " [" + record.level + "] " +
           "[" + record.threadName + "] " + record.message
```

JSONFORMATTER.FORMAT

```
format(record):
    return jsonEncode({
        "timestamp": record.timestamp,
        "level":     record.level,
        "thread":    record.threadName,
        "message":   record.message
    })
```

Both formats are illustrative. Real plain-text formats typically follow a configurable template like `"{timestamp} {level} [{thread}] {message}"` so the output shape can change without code changes. For interview scope, hardcoding is fine. Mention you'd accept a format string in the constructor for a real implementation if asked.

Sink implementations

`ConsoleSink` writes to stdout. `FileSink` writes to an opened file handle.

CONSOLESINK.WRITE

```
write(formatted):
    stdout.println(formatted)
```

FILESINK.WRITE

```
FileSink(filePath):
    this.fileWriter = openFile(filePath, mode = APPEND)

write(formatted):
    fileWriter.append(formatted + "\n")
```

The `FileSink` constructor opens the file once and keeps the handle. Don't reopen on every write. The open syscall is expensive, and reopening per call would dwarf the actual write cost

by orders of magnitude. The file gets closed when the application shuts down, or when you call an explicit `close()` if you want a graceful path.

Note both sinks delegate the actual byte-writing to the language's I/O primitive. Buffering, encoding, OS-level flushing - those are the underlying `OutputStream` or `FileWriter` 's concern. The sink's job is just to call the right method.

> In a real `FileSink` , you'd usually want explicit `flush()` after each write (or after some bounded interval) so the most recent log lines aren't sitting in an OS buffer when the process crashes. The default behavior in most languages is to flush on close, which is exactly the wrong moment for a logger - you want recent logs visible *before* the crash, not after a clean shutdown.

LogRecord

The constructor takes the four fields and stores them. Getters return them. That's the entire class.

LOGRECORD

```
LogRecord(timestamp, level, message, threadName):
    this.timestamp  = timestamp
    this.level      = level
    this.message    = message
    this.threadName = threadName

getTimestamp():  return timestamp
getLevel():      return level
getMessage():    return message
getThreadName(): return threadName
```

Complete Code Implementation

While most interviews only require pseudocode, some ask for working code. Below is a complete implementation in common languages for reference.

LogLevel.py | LogRecord.py | Formatter.py | PlainTextFormat | Python

```python
from enum import IntEnum


class LogLevel(IntEnum):
    DEBUG = 10
    INFO = 20
    WARN = 30
    ERROR = 40
    FATAL = 50
```

Verification

In an LLD interview, you always want to be proactive about verifying your design. Here's a quick check with three scenarios that should cover everything: one happy path with filtering, one with concurrent access, and one with a sink failure.

Scenario 1. Two destinations with different thresholds, single thread.

We have a console destination at `minLevel = DEBUG` and a file destination at `minLevel = WARN`. The application calls `logger.info("user logged in")`.

```
LOGGER.INFO('USER LOGGED IN')
```

```
Logger.log(INFO, "user logged in"):
    record = LogRecord(2026-05-06T10:00:00, INFO, "user logged in", "main")

    iterate destinations:
        consoleDestination.write(record):
            INFO >= DEBUG → continue
            formatted = "2026-05-06T10:00:00 [INFO] [main] user logged in"
            lock.acquire()
            consoleSink.write(formatted)  → line appears on stdout
            lock.release()

        fileDestination.write(record):
            INFO < WARN → silent drop, return

Result: console wrote the line, file ignored it.
```

The record reaches the console because INFO clears the DEBUG threshold; the file destination filters it out because INFO is below WARN. Each destination decided independently using its own threshold.

Scenario 2. Two threads, same destination, contended write.

There's a single file destination at `minLevel = WARN`, and Thread A and Thread B both call `logger.error("...")` at the same moment.

CONCURRENT LOG CALLS

```
Thread A:                                       Thread B:
log(ERROR, "A failed"):                         log(ERROR, "B failed"):
    record_A = LogRecord(t1, ERROR, ...)            record_B = LogRecord(t2, ERROR,
    fileDestination.write(record_A):                fileDestination.write(record_B)
        ERROR >= WARN → continue                        ERROR >= WARN → continue
        formatted_A = format(record_A)                  formatted_B = format(record
        // both threads have formatted in parallel — safe, formatter is pure

        lock.acquire()    ← Thread A wins
                                                        lock.acquire()    ← blocked
        sink.write(formatted_A)
        lock.release()
                                                        lock.acquire()    ← unblock
                                                        sink.write(formatted_B)
                                                        lock.release()
```

```
Result: both records hit the file in lock-acquisition order (A then B here).
       No interleaved bytes — the lock made each sink.write atomic relative to
       other sink.writes on the same destination.
```

The order on the wire reflects who got the lock first, not who called `log()` first. That's fine. The requirements only ask that writes don't interleave or corrupt output, not that they preserve global call order across threads, and the per-destination lock guarantees exactly that. If an application needs strict per-destination sequencing, the async-queue extension above is what gets you there.

Scenario 3. Sink failure, other destinations unaffected.

We have both a console destination and a file destination at `minLevel = DEBUG`, but the disk has filled up so `FileSink.write` throws `IOException` on every call. The application calls `logger.warn("disk space low")` — a logger trying to write a disk-full warning to the disk that's full.

LOGGER.WARN WITH FILESINK FAILING

```
Logger.log(WARN, "disk space low"):
    record = LogRecord(now(), WARN, "disk space low", "main")

    iterate destinations:
        consoleDestination.write(record):
            WARN >= DEBUG → continue
            formatted = format(record)
            lock.acquire()
            consoleSink.write(formatted)  → "disk space low" appears on stdout
            lock.release()

        fileDestination.write(record):
            WARN >= DEBUG → continue
            formatted = format(record)
            lock.acquire()
            try:    fileSink.write(formatted)  → throws IOException
            catch:  stderr.write("logger: sink write failed: disk full")
            finally: lock.release()
            // returns normally — exception did not escape Destination.write()

Result: console line written, file line dropped, one diagnostic line on stderr,
        application keeps running.
```

The console got the record. The file silently failed but emitted a stderr diagnostic so the failure isn't invisible. The application never saw an exception. `Logger.log()` finished its iteration normally because `Destination.write()` swallowed the failure inside its own try/catch. One destination's failure doesn't propagate to the others or to the caller.

Extensibility

If there's time left after implementation, interviewers often ask "what if" questions to see whether your design can evolve cleanly. You typically won't implement these changes, you'll just explain where they'd fit.

For a logger, the two most common follow-ups by far are async writes (the most visible production concern, and the one we explicitly scoped out in requirements) and hierarchical named loggers (every real framework has them, so anyone who's used Log4j, SLF4J, or Python's `logging` is going to ask). Other natural extensions like hot-reload of config, log rotation, and per-message deduplication windows all slot in cleanly without rewriting the core model. The two below are the ones worth walking through in detail.

1. "How would you make `log()` non-blocking?"

The current design holds a per-destination lock around `sink.write()`. That's correct, but it means a slow file write or a flaky network destination blocks the calling thread for as long as the I/O takes. For a logger that fires from tens of thousands of call sites, that adds up to real latency in the application.

> "I'd put a bounded **blocking queue** in front of each destination's sink. `log()` enqueues the record and returns immediately, and a dedicated worker thread per destination drains the queue and does the actual write. Concurrent producers, single consumer per resource, which means the consumer side doesn't even need a lock anymore."

DESTINATION WITH ASYNC WRITES

```
class Destination:
    - formatter, minLevel, sink
    - queue: BlockingQueue<LogRecord>    // bounded
    - worker: Thread
```

```
    + Destination(formatter, minLevel, sink, capacity):
        this.queue  = new BlockingQueue(capacity)
        this.worker = startThread(drain)

    + write(record):
        if record.level < minLevel: return
        queue.put(record)               // blocks if full (or drop / throw)

    - drain():                           // runs on the worker thread
        while running:
            record    = queue.take()
            formatted = formatter.format(record)
            sink.write(formatted)        // single consumer, no lock needed
```

This is what Log4j's `AsyncAppender` and Python's `QueueHandler` ship in production. The benefits: caller latency drops to "enqueue an object," each destination gets strict per-destination sequencing for free (single consumer pulls from a FIFO queue), batching is now possible if the worker pulls in chunks, and the bounded queue gives you an explicit place to define a backpressure policy.

It's not free, though. A good interviewer will push on three things.

Worker lifecycle. Each destination now owns a thread that runs for the life of the application. Shutdown is the tricky part — you have to signal the worker to stop, drain whatever's already in the queue, and wait for it to finish before the process exits. Skip that and you lose every record that was buffered when the JVM (or whatever) died.

Overflow policy. A bounded queue forces a decision about what happens when it fills up. You've got four reasonable choices and each one is wrong for some workload. `block` the producer (which defeats the whole point of going async), `drop` the new record (which silently loses data right when something's going wrong), drop the oldest record (same problem, different end of the queue), or `throw` an exception (which turns logging back into something callers have to catch). Most production loggers default to drop-newest with a stderr diagnostic, but the right answer depends on whether you care more about not losing records or not blocking callers.

Debuggability. The actual write now happens on a different thread than the call site. A stack trace at the moment of an I/O failure no longer points back to the code that emitted the record,

which makes "why did this log line fail to write?" meaningfully harder to answer. You can mitigate by logging the record's call-site info into the diagnostic, but the gap is real.

> Worth flagging in the interview that "async writes" and "thread-safe writes" solve different problems. The lock is about correctness (no interleaved bytes on the wire). The queue is about coordination (don't block the caller). The two compose cleanly. A single-consumer queue per destination actually lets you drop the lock in this exact design, since only the worker thread ever touches the sink. You'd add it back the moment two destinations share an underlying stream, but for the common case the queue is enough on its own.

2. "How would you support hierarchical named loggers?"

Production loggers don't construct one global `Logger` and pass it around. They expose `LoggerFactory.getLogger("com.app.service.payments")`, and the returned logger inherits configuration from its parent in the dotted-name tree. The team that owns `com.app.service` can set its threshold once and have everything underneath pick it up unless overridden. If the candidate has used Log4j, SLF4J, or Python's `logging`, they'll recognize this immediately.

> "I'd add a `name` field and a `parent` pointer to `Logger`, and put a `LoggerFactory` in front of construction. The factory keeps a registry keyed by name, looks up the parent from the dotted prefix, and falls back to the root when none exists. Effective level and effective destinations walk the parent chain when they're not set on the logger itself."

That's enough to land the question. Two things worth naming if asked:

- **Caching.** `log()` is the hottest path in the system. Real frameworks cache the effective level on each `Logger` and invalidate it when configuration changes, instead of walking the parent chain on every call.
- **The factory is a deliberate global.** The whole point of the registry is that two callers anywhere in the codebase asking for `getLogger("com.app.service")` get back the same instance. That's the rare case where shared application state is the requirement, not an accident.

What is Expected at Each Level?

So, as an interviewer, what am I looking for at each level?

Junior

At the junior level, I'm checking whether you can break a one-sentence prompt into a working object model. You should land on a `Logger` that owns a list of destinations, an immutable `LogRecord` carrying timestamp, level, message, and thread name, and a `LogLevel` enum with the five required values. Your `log()` method should build a record and fan it out to every destination, and each destination should drop records below its threshold before writing. Basic error handling matters: a null message shouldn't crash, an out-of-range level shouldn't either. It's fine if your first cut couples format and destination together (a `JsonFileDestination` class, for example) and you only see the 2D explosion problem after I push on it.

Mid-level

For mid-level candidates, I expect the format-vs-destination split without much guidance. You should reach for composition the moment you hear "any format with any target" and pull `Formatter` into its own interface, and you should recognize that one `Destination` class composing a formatter and a write target beats N×M subclasses. You should immediately see that `LogRecord` deserves to be its own immutable type so that adding a logger name or request ID later doesn't ripple through every method signature. I don't expect you to nail concurrency, but we should be able to have a conversation about it, and my hints should enable you to arrive at a reasonable solution.

Senior

Senior candidates should produce a design that demonstrates systems thinking. The class boundaries should be obvious without deliberation, and you should proactively walk through the inheritance-vs-composition tradeoff for `Destination` instead of waiting for me to ask. I expect you to land on a `Sink` interface (or call out the inheritance variant and explain why you'd still pick composition) and articulate why the high-level workflow class shouldn't depend on `ConsoleSink` or `JsonFormatter` directly. You should catch the time-of-capture decision yourself: that timestamp and thread name belong at the top of `Logger.log()` so every destination sees the same moment, not inside `Destination.write()`. On concurrency, you should reach per-destination locks without prompting, explain why a global lock on `log()` is

the wrong default (a slow file write blocking instant console output), and call out that `format` happens outside the critical section because records are immutable and formatters are pure functions. On failure handling, you should propose swallowing per-destination exceptions with a stderr diagnostic so the logger never crashes the caller but doesn't fail silently either. Strong candidates finish early and can discuss how the design evolves for async writes (bounded queue per destination, overflow policy, worker lifecycle on shutdown) and hierarchical named loggers (the registry as the one defensible global, additivity along the parent chain, caching the effective level on the hot path).

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

Start Quiz